

LABORATORY 3

Widgets & UI

composing layouts and handling gestures

What you'll build today



Compose a screen

Build a login screen from Column, Row, Padding and Expanded, not one giant build()



Split it into widgets

Extract reusable StatelessWidget and StatefulWidget components with clear parameters



Handle gestures

Make list rows respond to swipes, with a destructive action and a way back



Lecture 3, applied

This lab puts Lecture 3 to work: the widget tree, composition and rebuilds

Where your code goes

Same repository, same Flutter project as Labs 1 and 2. Create a branch named lab3.

Task I goes in lib/screens/login_screen.dart, Task II in lib/screens/todo_screen.dart.

Every component you extract gets its own file under lib/widgets/.

Point main.dart at whichever screen you are currently working on.

Keep the todo app. Lab 4 attaches DevTools to this exact app instead of rebuilding it.

```
lib/  
  main.dart  
  screens/  
    login_screen.dart  
    todo_screen.dart  
  widgets/  
    labeled_field.dart  
    todo_tile.dart
```

No pixel-perfect copy of anyone's app. You are given the element list; the styling decisions are yours.

Stateless or stateful?

StatelessWidget

Holds no mutable state of its own; everything it draws arrives through its constructor.

It does not “build once”: it rebuilds whenever its parent rebuilds or a dependency changes.

Use it for the logo block, a labeled field, the primary button, a single todo row.

StatefulWidget

Owns state that changes over its lifetime: controllers, a password-visibility flag, the todo list.

setState() marks that subtree dirty; only dirty subtrees rebuild, never the whole tree.

The screens themselves are stateful here; most of what they contain does not need to be.

Start stateless. Promote a widget to stateful only when it has to remember something between builds.

The layout widgets you need today

Column

children stacked vertically; mainAxisAlignment
spaces them out

Row

children side by side; the horizontal equivalent of
Column

Expanded

lets a child take the remaining space inside a Row
or Column

Padding

space around a single child; EdgeInsets says how
much

SizedBox

a fixed width or height; the plain way to add a gap

Center

centers its child inside the space the parent gave it

Container

padding, margin, color, border and size in one
widget

Stack

children drawn over each other; Positioned places
them

SingleChildScrollView

makes a Column scroll when the keyboard shrinks
the screen

Every one of these takes children, not pixels. You describe the arrangement; the framework does the measuring.

Login screen layout

lib/screens/login_screen.dart. The required elements are listed on the next slide.

1. Build a login screen containing every element listed on the following slide.
2. Lay it out with Column, Padding, SizedBox and Expanded, with no absolute positioning.
3. Break the screen into smaller reusable components instead of one long build() method.
4. Make each component that only displays data a StatelessWidget.
5. Make the screen itself a StatefulWidget, so it can hold the field controllers and the error text.
6. Show an error message under the fields when either field is empty on submit.
7. Design the visual style yourself: colors, spacing and typography are your decisions.

Hints

TextField + TextEditingController

obscureText: true for the password

ElevatedButton / TextButton

InputDecoration(labelText:, hintText:)

const constructors + required params

Done when

Every element on the list is on screen and readable

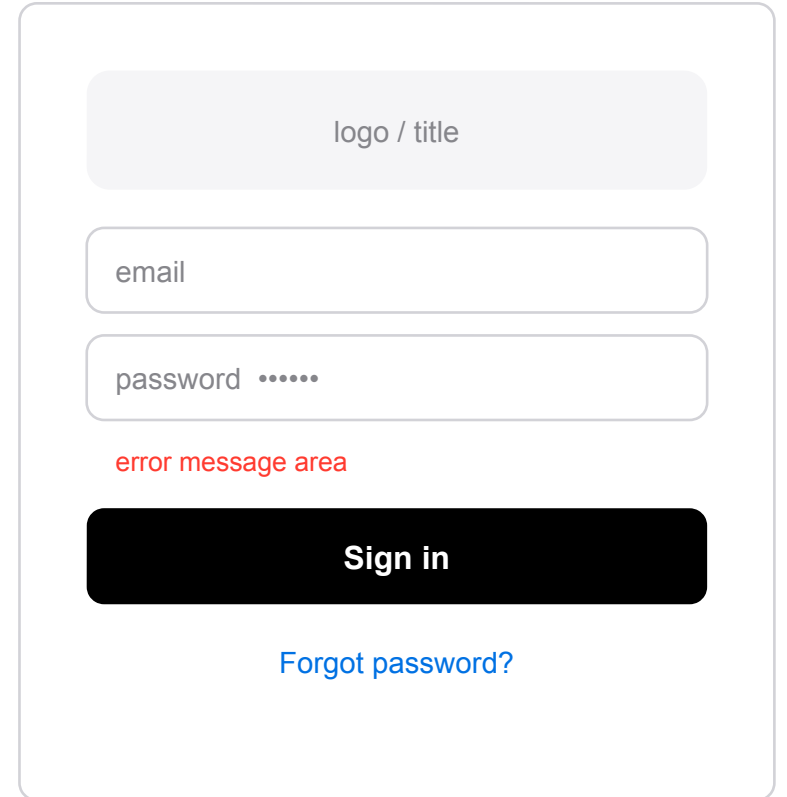
At least two components live in lib/widgets/

Submitting an empty form shows the error text

TASK 1

The screen, element by element

Logo or title	An icon and the app name at the top. Anything that identifies the app.
Email field	A labeled text input, keyboardType: TextInputType.emailAddress.
Password field	A labeled text input with obscureText: true.
Error area	A reserved line under the fields for one message. Empty when all is well.
Primary action	A full-width “Sign in” button that reads both controllers.
Secondary link	“Forgot password?” as a TextButton, not a second big button.
Spacing	Consistent padding around the form, and a scroll view around the Column.

A wireframe of a login screen. It features a light gray rounded rectangle at the top containing the text "logo / title". Below this are two white rounded rectangles for "email" and "password". A red text label "error message area" is positioned below the password field. At the bottom is a black rounded rectangle with the text "Sign in" in white, and a blue text link "Forgot password?" below it.

logo / title

email

password

error message area

Sign in

Forgot password?

A wireframe, not a design.

What “reusable component” means

```
// lib/widgets/labeled_field.dart
class LabeledField extends StatelessWidget {
  const LabeledField({super.key, required this.label,
    required this.controller, this.obscure = false});
  final String label;
  final TextEditingController controller;
  final bool obscure;

  @override
  Widget build(BuildContext context) => TextField(
    controller: controller,
    obscureText: obscure,
    decoration: InputDecoration(labelText: label));
}
```

Why this counts

It renders from its parameters only, which is what makes it stateless.

required names what the caller must pass;
obscure has a default, so most callers ignore it.

The email and password fields become two calls to the same widget.

A const constructor with super.key lets Flutter skip rebuilding it when nothing changed.

Todo list with swipe gestures

lib/screens/todo_screen.dart. Lab 4 debugs this app, so build it carefully.

1. Create a Todo model with at least an id, a title and a done flag.
2. Hold a list of todos in the screen's State and render it with ListView.builder.
3. Render each todo as a Card containing a ListTile.
4. Wrap each row in a Dismissible with a stable, unique key.
5. Swipe left to delete the todo, removing it from the list.
6. Swipe right to mark the todo as completed, keeping the row on screen.
7. Give each swipe direction its own colored background with an icon, so the gesture is discoverable.
8. Show a SnackBar with an Undo action after a delete.

Hints

Dismissible: the swipe widget

key: ValueKey(todo.id)

direction: DismissDirection....

confirmDismiss, onDismissed

ScaffoldMessenger.of(context)

Done when

Both swipe directions work on every row

Deleting the top row does not remove the wrong one

Undo restores the deleted todo

The anatomy of one swipeable row

```
// one row of your todo list
ListView.builder(
  itemCount: todos.length,
  itemBuilder: (context, i) => Dismissible(
    key: ValueKey(todos[i].id),
    background: const ColoredBox(color: Colors.green),
    secondaryBackground: const ColoredBox(color: Colors.red),
    confirmDismiss: (direction) async =>
      direction == DismissDirection.endToStart,
    onDismissed: (_) => setState(() => todos.removeAt(i)),
    child: Card(child: ListTile(title: Text(todos[i].title))),
  ),
)
```

The four common mistakes

key is not optional. Without a stable unique key the wrong row disappears.

background is the swipe-right layer, secondaryBackground the swipe-left one.

Returning false from confirmDismiss keeps the row, which is how “completed” works without deleting.

onDismissed must remove the item from your list, or the next build throws.

Layout errors and their fixes

A `RenderFlex` overflowed by 137 pixels on the bottom.

The `Column` is taller than the screen. Wrap it in `SingleChildScrollView`, or make one child `Expanded`.

Vertical viewport was given unbounded height.

A `ListView` inside a `Column` with no height. Wrap the `ListView` in `Expanded`.

A dismissed `Dismissible` widget is still part of the tree.

`onDismissed` did not actually remove the item from the list backing the builder.

The wrong row disappears after a swipe.

Two rows share a key, or the key is the list index. Key by the `todo`'s own id.

The keyboard covers the password field.

Put the form inside a `SingleChildScrollView`; `Scaffold` then resizes around the keyboard.

How to hand this in

Everything goes to your GitHub repository. No Word documents, no email, no Teams upload.

Commit your work to a branch named lab3, open a pull request, and merge it once it works.

Screenshots asked for in a task go in a /screenshots folder in the same repository.

Your commit history is part of the assessment. Commit as you go, not once at the end.

Lab 4 starts from this code, so make sure the app still builds after your pull request is merged.

Checklist

- code committed and pushed
- screenshots where asked
- app builds and runs
- pull request merged

Wrapping up



Recap

A screen is a tree of small widgets, not one long `build()`

Stateless until the widget has to remember something

Dismissible turns a list row into a gesture target

Keys are what tell Flutter which row is which



Before next lab

Merge your lab3 pull request

Keep the todo app; Lab 4 debugs this exact app with DevTools

Skim the DevTools overview page before the session



Read more

docs.flutter.dev/ui/layout

docs.flutter.dev/ui/interactivity

api.flutter.dev · `Dismissible`, `ListView.builder`